RSA Encryption, Quantum Computers and Shor's Algorithm

Menase Teshale

RSA Algorithm for Asymmetric Encryption

Asymmetric encryption is a type of encryption where there are two keys called *public* and *private* where one(typically the private key) can only decrypt and the other(public key) can only encrypt the messages. This solves the middleman problem with symmetric encryption wherein you have to send an unencrypted key to begin communication but if the key is intercepted then the rest of the communication being encrypted is of no importance. Asymmetric encryption is vital for there to be any secure connection to the internet. It is also used in digital signatures and authentication for signing documents and commits on github, asymmetric encryption *is the reason you can be sure code for your operating system is written by the people whose job it is to write it.* The **RSA** algorithm is one of the simplest asymmetric encryption algorithms and it works as follows:

- Choose two prime numbers p, q then set $N = p \times q$
- Find $\phi(N) = (p-1) \times (q-1)$
- Find e such that $gcd(e, \phi) = 1$
- Find d such that $e \times d \equiv 1 \ mod \ \phi$ that is the multiplicative inverse of $e \ mod \ \phi$
- $C \equiv M^e \ mod \ N$ therefore (N, e) is the public key
- $M \equiv C^d \ mod \ N$ therefore (N, d) is the private key

Breaking RSA on Classical Computer

This can be done by first selecting two random primes and checking if they multiply to give N if they do you can input them into the second step of the RSA algorithm to get the private and public keys. This is needlessly time consuming as you actually dont need to find the numbers themselves. You only need to find a multiple of p or q **but not both** then all you have to do is $gcd(a, N)$ to find one of the primes (say p) then you can easily find the other $q = N/p$. If you didn't find a multiple of p *or* q a result in number theory guarantees that there is a number g for your number a such that $a^g = mN + 1$ for some m. We can rearrange this equation to get $(a^{\frac{g}{2}} - 1) \times (a^{\frac{g}{2}} + 1) = mN$. Now we are not guaranteed either of these factors is a number we need one of them could be a multiple of N while the other one a factor of m giving us no new information. But it is much more likely that these factors are the numbers we need 37. 5% of such pairs to be exact. But the difficulty is in finding the original g to do such factorization. On classical computers we are back now to sequential guessing, this is where quantum computers come in.

Quantum Computers

Are a special type of computer that can in a single step perform a number of

computations that doesn't scale linearly to the number of gates/cores/pipeline phases used to build it. They do this by using the property of superposition where a *qbit* can be in a combination of multiple states at the same time. It has to be noted that the qbit will when observed land on one of these states with a given probability and then the quantum properties (entanglement, superposition...) are lost. Quantum computers can only answer one question very fast not multiple questions at the same time. The important properties of quantum computers are:

- Superposition - Used to represent a combination of multiple states
- Interference - The basic concept of a computational step where correct answers reinforce while incorrect answers cancel out
- Entanglement - When a state cannot be decomposed cleanly, meaning that a part of the system affects another part of the system

Breaking RSA on Quantum Computers (Shor's Algorithm)

To break RSA on a quantum computer we first represent our state a s a superposition of all the guesses for p. $|1\rangle + |2\rangle + |3\rangle \ldots$ then you from this create a superposition which for each guess has $|x, a^x mod N\rangle$ now if you measure only the remainder part it collapses on a single value *but the other half of the superposition doesn't.* This is one of the properties of a quantum system if measured by part the other part remains in a superposition but one in which the measured value is guaranteed to be the same for all components of the superposition. This is why we kept all the quesses in the previous step since all the values of x that yield constant $a^x \ mod \ N$ are separated by a period of g the number which when given our number which is unlikely to share factors with N gives us a pair which are likely to. Now if we input this superposition into a system called the quantum forier transform the remaining state will be the frequency of the superposition $\left|\frac{1}{g}\right\rangle$ and so our result is found in a few sequential steps.

Post Quantum Security

In the first segment we have hilighted how critical asymmetric encryption is so if quantum computers can crack one of the most widely used asymmetric encryption schemes, is all lost? First of all there is no proof that all encryptions cannot be broken effectively by quantum or even classical computers. The best we have are conditional statements on what seem to be highly likely conjenctures like the **PRG** conjecncture that have held up in practical expirience. But there have been encryption schemes that have proven effective on both classical and quantum computers based on linear algebra and the concept of lattices.

Lattice-Based Cryptography

Lattice-based cryptography is one of the leading approaches to *post-quantum cryptography*. Unlike RSA, whose security depends on the difficulty of factoring large integers, lattice-based schemes rely on problems involving high-dimensional geometric structures called **lattices**. A lattice can be thought of as an infinite grid of points generated by combining a small set of vectors with integer coefficients. Although generating points on such a grid is easy, solving certain computational problems on these grids is believed to be difficult for both classical and quantum computers.

Two important lattice problems are the *Shortest Vector Problem (SVP)* and the *Learning With Errors (LWE)* problem. While the details are mathematically involved, the key idea is that finding hidden information inside a high-dimensional lattice appears to require an exponential amount of work. No efficient classical or quantum algorithm comparable to Shor's algorithm is known for solving these problems.

Learning With Errors

The intuition behind the Learning With Errors problem is surprisingly simple. Suppose someone chooses a secret vector s and publishes many equations of the form

$$b = As + e$$

where A is public, b is public, and e is a small random error called *noise*. If there were no error term, recovering s would simply require solving a system of linear equations, which is computationally easy. The addition of even a small amount of random noise completely changes the problem. The equations no longer have an exact solution, and determining the original secret becomes computationally difficult.

The legitimate receiver knows additional information that allows the small errors to be removed or tolerated during decryption. An attacker, however, only sees many noisy equations and must determine which part is genuine information and which part is random error. This distinction appears to remain difficult even for quantum computers.

Why Shor's Algorithm Does Not Apply

Shor's algorithm is remarkably effective because it exploits the periodic structure present in problems such as integer factorization and the discrete logarithm problem. By using quantum superposition together with the Quantum Fourier Transform, it efficiently discovers this hidden period, allowing RSA and several other public-key cryptosystems to be broken.

Lattice-based cryptography deliberately avoids exposing such periodic structure. The random noise added to each equation destroys the exact regularity that Shor's algorithm depends upon. Instead of searching for a clean repeating pattern, an attacker is faced with distinguishing a hidden signal from random perturbations in a very high-dimensional space. The Quantum Fourier Transform no longer isolates a useful period because there is no exact period to recover.

This does not constitute a mathematical proof that lattice-based cryptography is secure against quantum computers. Rather, it means that no quantum algorithm analogous to Shor's algorithm is currently known for solving the underlying lattice problems efficiently. As a result, these schemes are considered strong candidates for securing communications in the presence of large-scale quantum computers.

Current State of Post-Quantum Cryptography

Research into post-quantum cryptography has accelerated as quantum hardware continues to improve. Rather than waiting until practical quantum computers exist, cryptographers are standardizing algorithms that can replace RSA and elliptic curve cryptography before they become vulnerable.

Several standardized encryption and digital signature schemes are based on lattice problems such as Learning With Errors and its variants. These schemes generally require larger public keys and ciphertexts than RSA, but they provide security assumptions that are currently believed to withstand attacks from both classical and quantum computers.

Although no cryptographic system can be guaranteed secure forever, lattice-based cryptography represents one of the strongest known defenses against quantum attacks. By replacing algebraic structures that exhibit exploitable periodicity with noisy high-dimensional lattice problems, these schemes avoid the fundamental weakness that allows Shor's algorithm to efficiently break RSA.